

Lab 24 - Expected verification output

Run from the lab directory:

```
bash verification/checks.sh
```

The script takes about 15 seconds - it deliberately sleeps 12 s between two scrapes to prove the counter increases.

Honesty note - what this script does and does not assert

checks.sh never asserts a specific metric value, because no metric value in this lab is predictable. It asserts only things that are true by construction:

- * Structural invariants of the exposition format - a # HELP and # TYPE line per family, the mandatory le="+Inf" bucket, and the requirement that le="+Inf" equals _count. That last one is a rule of the format, so it is safe to check exactly.
- * Direction of change - the counter must be strictly greater on the second scrape than the first. Which numbers appear is irrelevant.
- * Query shape - that rate() returns a non-empty result and that histogram_quantile() does not return NaN.

The one quantitative claim in the lab (rate() should land between 0.7 and 1.1 per second) is derived from the generator's arithmetic in data/generate-metrics.sh, which adds 7-11 to the counter every 10 seconds. You verify that yourself in Step 5.2; the script does not, because the value drifts at the edges of the scrape window.

Fully passing run

```

== Namespace isolation
PASS namespace kcna-lab24 exists
PASS Pod Security Admission enforce=baseline

== Part 2 - ConfigMaps sourced from data/
PASS configmap/depot-metrics-generator exists
PASS configmap/depot-metrics-generator key 'generate-metrics.sh' is populated
PASS configmap/prometheus-config exists
PASS configmap/prometheus-config key 'prometheus.yml' is populated

== Part 2 - scrape target workload
PASS deployment/depot-metrics-sim exists
PASS replicas=1 (required: scraping through a Service with >1 replica makes counters jump backwards)
PASS container 'generator' present
PASS container 'exporter' present
PASS all target images pinned (busybox:1.36 nginx:1.27-alpine)

== Part 3 - exposition format served over HTTP
PASS pod/metrics-client exists
PASS /metrics returns a non-empty body
PASS # HELP line present for the counter
PASS # TYPE declares the counter
PASS # TYPE declares the gauge
PASS # TYPE declares the histogram
PASS histogram has the mandatory le="+Inf" bucket
PASS le="+Inf" (97) equals _count (97) as the format requires
PASS _info pattern present (value 1, labels carry the data)
PASS counter increased across two scrapes (97 -> 106)

== Part 4 - Prometheus and its targets
PASS deployment/prometheus exists
PASS prometheus image is pinned (prom/prometheus:v2.54.1)
PASS --web.enable-lifecycle set (needed for the Part 6 hot reload)
PASS prometheus is Ready
PASS job 'depot-portal' is configured
PASS job 'prometheus' (self-scrape) is configured
PASS no targets are DOWN
PASS PromQL query API responds
PASS rate() over the counter returns a non-empty result
PASS histogram_quantile(0.95, ...) returns a value

== Worksheet
PASS worksheet present at data/exposition-worksheet.csv
PASS worksheet header intact

== Scope guard
PASS no Prometheus Operator CRDs - this lab is deliberately Operator-free
PASS no lab-24 objects in kube-system

SUMMARY: 33 passed, 0 failed, 0 skipped

```

Exit code 0. The two numbers shown in `le="+Inf" (97) equals _count (97)` and `counter increased across two scrapes (97 -> 106)` will differ on your run - only the relationship matters.

Common partial runs

Run immediately after Step 4.4 - not enough history yet

```
SKIP rate() empty - allow ~2 minutes of scrape history after Step 4.4, then re-run
SKIP histogram_quantile empty - needs ~5 minutes of history; re-run later
```

SUMMARY: 31 passed, 0 failed, 2 skipped

Exit code 0. This is not a defect. `rate()` needs at least two samples inside its range window, and `histogram_quantile` over `[5m]` needs the window to have filled. Finish Part 5 and re-run.

Run in the middle of Part 6, after the injection

```
FAIL 1 target(s) DOWN - if you are mid-Part-6 this is expected; finish Step 6.8
```

Exit code 1. Expected mid-lab: you have deliberately pointed the scrape config at port 9101. Complete Step 6.8 to restore the good config, reload, and re-run.

Prometheus OOMKilled

```
FAIL prometheus not Ready yet - wait and re-run
```

Check for the real cause:

```
kubectl get pod -n kcna-lab24 -l app=prometheus \
-o jsonpath='{.items[0].status.containerStatuses[0].lastState.terminated.reason}{"\n"}'
```

If this prints `OOMKilled`, the memory limit is too small. The supplied manifest requests 128Mi and limits 512Mi for exactly this reason - a TSDB is not a 50m/64Mi workload. See the resource comment in `manifests/20-prometheus.yaml`.

histogram_quantile returns NaN

```
FAIL histogram_quantile returned NaN - check that 'le' survives the sum by clause
```

The `le` label was aggregated away. The inner clause must be `sum by (le)`
`(rate(depot_portal_scan_latency_seconds_bucket[5m]))`.

Manual observations the script cannot make for you

#	Observation	Where
1	Raw exposition text fetched with wget, before Prometheus is involved	Step 3.2
2	A counter rising while a gauge moves in both directions, across two scrapes	Step 3.5
3	Buckets non-decreasing as le rises, and le="+Inf" equal to _count	Step 3.7
4	Content-Type: text/plain; version=0.0.4; charset=utf-8	Step 3.9
5	The instance label with no :9100 - proof the relabel_configs rule fired	Step 5.1
6	A measured rate() inside the predicted 0.7-1.1/s band	Step 5.2
7	rate(...[5s]) returning empty because the window is narrower than the scrape interval	Step 5.3
8	up{job="depot-portal"} flipping to 0 after one wrong port number	Step 6.5
9	lastError reading connect: connection refused	Step 6.6
10	The gauge going stale (empty), not to zero, during the outage	Step 6.7
11	History surviving the outage, so the gap is visible after recovery	Step 6.9